GUÍA DE INSTALACIÓN Y CONFIGURACIÓN

Django Moderno + MongoDB Backend Oficial con Django Admin

Proyecto:	djangopractica
Tecnologías:	Django 6.1, MongoDB, django-mongodb-backend, PyMongo 4.18
Base de Datos:	MongoDB Local (puerto 27017) -> djangopractica_db
Panel Administrativo:	Habilitado y 100% funcional con ObjectId nativo

1. Introducción y Selección de Arquitectura

Tradicionalmente, en tutoriales antiguos de Django y MongoDB se utilizaba la librería 'djongo'. Sin embargo, 'djongo' quedó abandonada y solo es compatible con versiones obsoletas de Django (hasta la 3.1/3.2). Al intentar usarla con Django moderno se presentan fallos críticos de sintaxis SQL y migraciones.

Por esta razón, MongoDB lanzó el soporte oficial: django-mongodb-backend. Este paquete fue diseñado específicamente para las versiones modernas de Django (5.x y 6.x) y permite:

• Integrar el ORM estándar de Django con colecciones de MongoDB sin emular SQL defectuoso.

• Usar ObjectIdAutoField nativo de MongoDB en lugar de IDs numéricos relacionales.

• Soporte total para el panel de administración (Django Admin), sistema de usuarios y sesiones.

> **Compatibilidad Garantizada**
> Regla de versiones: El paquete django-mongodb-backend sigue la misma numeración que Django. Por ejemplo, para Django 6.1.x se instala django-mongodb-backend==6.1.*.

2. Paso a Paso Completo con Comandos

Paso 1: Creación del Entorno Virtual (venv)

Para mantener el proyecto aislado y evitar conflictos entre versiones de paquetes globales, se crea y activa un entorno virtual en la raíz del proyecto.

Creación y activación de entorno virtual

```
# 1. Abrir terminal en la carpeta del proyecto
cd "c:\Users\Dell Latitude i7 8th\Desktop\djangopractica"

# 2. Crear el entorno virtual llamado .venv
python -m venv .venv
```

```
# 3. Activar el entorno virtual en PowerShell:
.\.venv\Scripts\Activate.ps1

# (Si usas CMD tradicional, ejecuta: .\.venv\Scripts\activate.bat)
```

Paso 2: Instalación de Dependencias Necesarias

Instalamos directamente el conector oficial de MongoDB para Django. Pip resuelve e instala automáticamente la versión compatible de Django (6.1.x), el driver pymongo y utilidades asociadas.

📌 **Instalación del conector oficial**

```
.\.venv\Scripts\pip install django-mongodb-backend
```

Las dependencias instaladas quedan registradas en requirements.txt: django==6.1.1, django-mongodb-backend==6.1.0, pymongo==4.18.2, dnspython, sqlparse, asgiref, tzdata.

Paso 3: Creación del Proyecto Django con la Plantilla Oficial

MongoDB Labs proporciona una plantilla oficial para django-admin startproject que viene preconfigurada con las migraciones adaptadas para MongoDB (carpeta mongo_migrations) y las clases de configuración para Admin y Auth.

📌 **Generar proyecto con plantilla oficial MongoDB**

```
.\.venv\Scripts\django-admin.exe startproject config --template
https://github.com/mongodb-labs/django-mongodb-
project/archive/refs/heads/6.0.x.zip .
```

Nota: El punto final (.) indica que el archivo manage.py y la estructura se creen directamente en la carpeta actual sin anidar carpetas innecesarias.

Paso 4: Creación de la Aplicación de Django (miapp)

Creamos la aplicación interna (en este caso llamada miapp) utilizando la plantilla para aplicaciones de MongoDB, la cual define correctamente el default_auto_field como ObjectIdAutoField.

📌 **Generar la aplicación miapp**

```
.\.venv\Scripts\python.exe manage.py startapp miapp --template
https://github.com/mongodb-labs/django-mongodb-app/archive/refs/heads/5.2.x.zip
```

Paso 5: Configuración de config/settings.py

En el archivo config/settings.py se realizaron los ajustes clave para MongoDB y Django Admin:

Fragmentos clave en config/settings.py

```
# 1. Clave primaria global de MongoDB
DEFAULT_AUTO_FIELD = 'django_mongodb_backend.fields.ObjectIdAutoField'

# 2. Aplicaciones instaladas con compatibilidad para Admin y Auth
INSTALLED_APPS = [
    'config.apps.MongoAdminConfig',          # Reemplaza 'django.contrib.admin'
    'config.apps.MongoAuthConfig',           # Reemplaza 'django.contrib.auth'
    'config.apps.MongoContentTypesConfig',   # Reemplaza
'django.contrib.contenttypes'
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'django_mongodb_backend',                # Backend oficial MongoDB
    'miapp.apps.MiappConfig',                # Tu aplicación
]

# 3. Conexión a la base de datos MongoDB
DATABASES = {
    'default': {
        'ENGINE': 'django_mongodb_backend',
        'HOST': 'mongodb://localhost:27017/',
        'NAME': 'djangopractica_db',
    },
}
DATABASE_ROUTERS = ['django_mongodb_backend.routers.MongoRouter']

# 4. Módulos de migración compatibles con MongoDB
MIGRATION_MODULES = {
    'admin': 'mongo_migrations.admin',
    'auth': 'mongo_migrations.auth',
    'contenttypes': 'mongo_migrations.contenttypes',
}

# 5. Hosts permitidos e Idioma
ALLOWED_HOSTS = ['*']
LANGUAGE_CODE = 'es-es'
TIME_ZONE = 'UTC'
```

Paso 6: Definición del Modelo y Registro en el Admin

En miapp/models.py se define el modelo Articulo. Al heredar de models.Model, Django y django-mongodb-backend gestionarán automáticamente el identificador _id como un ObjectId de MongoDB:

miapp/models.py

```
# miapp/models.py
from django.db import models

class Articulo(models.Model):
    titulo = models.CharField(max_length=200, verbose_name='Título')
    descripcion = models.TextField(verbose_name='Descripción', blank=True)
    creado_en = models.DateTimeField(auto_now_add=True, verbose_name='Fecha de
Creación')
    activo = models.BooleanField(default=True, verbose_name='Activo')

    class Meta:
        verbose_name = 'Artículo'
        verbose_name_plural = 'Artículos'

    def __str__(self):
        return self.titulo
```

En miapp/admin.py se registra el modelo para que aparezca en el Django Admin con filtros y búsqueda:

📌 miapp/admin.py

```
# miapp/admin.py
from django.contrib import admin
from .models import Articulo

@admin.register(Articulo)
class ArticuloAdmin(admin.ModelAdmin):
    list_display = ('id', 'titulo', 'activo', 'creado_en')
    search_fields = ('titulo', 'descripcion')
    list_filter = ('activo', 'creado_en')
```

Paso 7: Ejecución de Migraciones hacia MongoDB

Django genera los archivos de migración y los aplica directamente en las colecciones de MongoDB:

📌 Comandos de migración

```
# Crear archivo de migración para miapp
.\.venv\Scripts\python.exe manage.py makemigrations

# Aplicar todas las migraciones a MongoDB (admin, auth, contenttypes, miapp,
sessions)
.\.venv\Scripts\python.exe manage.py migrate
```

Al completarse, se crean las colecciones en MongoDB: auth_user, django_session, miapp_articulo, django_admin_log, etc.

Paso 8: Creación del Superusuario para el Admin

Para poder ingresar al panel en /admin/ se genera un usuario administrador:

Creación del Administrador

```
# Modo interactivo (te solicita usuario, correo y clave):
.\.venv\Scripts\python.exe manage.py createsuperuser

# O modo rápido mediante shell:
.\.venv\Scripts\python.exe manage.py shell -c "from django.contrib.auth import
get_user_model; User = get_user_model(); User.objects.create_superuser('admin',
'admin@example.com', 'admin1234')"
```

Acceso al Panel Admin

Credenciales creadas en tu proyecto:

- Usuario: admin
- Contraseña: admin1234

Paso 9: Ejecutar el Servidor de Desarrollo

Para levantar el servicio web local y poder consultar el proyecto en el navegador:

Iniciar servidor Django

```
.\.venv\Scripts\python.exe manage.py runserver
```

Enlaces de acceso:

- Panel de Administración: http://127.0.0.1:8000/admin/
- Página Principal (Lista de artículos): http://127.0.0.1:8000/

3. Solución de Problemas Frecuentes

Problema 1: A) Error '127.0.0.1 rechazó la conexión (ERR_CONNECTION_REFUSED)':

Causa: El comando 'python manage.py runserver' no está ejecutándose en la terminal o la ventana se cerró.
Solución: Abre PowerShell, activa el entorno virtual y ejecuta de nuevo 'python manage.py runserver'.

Problema 2: B) Error de conexión con MongoDB (ServerSelectionTimeoutError):

Causa: El servicio local de MongoDB (mongod) está detenido.
Solución: Abre Servicios de Windows (services.msc) e inicia el servicio 'MongoDB Server', o ejecuta 'mongod' en terminal.

Problema 3: C) Error de permisos al activar venv en PowerShell ('La ejecución de scripts está deshabilitada'):

Solución: Ejecuta en PowerShell como Administrador:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

4. Tabla Resumen de Comandos Rápidos

Acción	Comando
1. Activar Entorno Virtual	`.\.venv\Scripts\Activate.ps1`
2. Instalar Paquetes	`pip install django-mongodb-backend`
3. Crear Migraciones	`python manage.py makemigrations`
4. Aplicar Migraciones a MongoDB	`python manage.py migrate`
5. Crear Superusuario	`python manage.py createsuperuser`
6. Iniciar Servidor	`python manage.py runserver`